

Freelance Client Onboarding & Proposal Agent

Executive Report

1. Business Goal

The goal of this project is to automate the initial stages of freelance client onboarding and proposal preparation. Freelancers and small development teams often spend significant time reviewing client requirements, matching them to available services, checking budget and deadline feasibility, and writing proposals.

The proposed agent system reduces this manual effort by using multiple specialized AI agents to analyze a client request, assess project scope, and generate a professional proposal. The system is designed as an assistant rather than a fully autonomous decision-maker. A human approval checkpoint remains in the workflow before a proposal can be finalized.

The system uses a verified service database containing service categories, base prices, and estimated delivery times. This provides a controlled source of information for project assessment.

2. System Architecture

The system follows a sequential multi-agent architecture:



The system contains three CrewAI agents:

1. Research Agent

- Identifies the most relevant freelance service.
- Extracts the client's stated requirements.
- Matches the request against verified service information.
- Identifies missing information.

2. Scope Analyst Agent

- Determines the estimated project price.
- Compares the price with the client's budget.
- Determines the estimated delivery time.
- Compares the timeline with the client's deadline.
- Identifies scope concerns.

3. Proposal Writer Agent

- Converts the verified research and scope analysis into a professional proposal.
- Produces sections for project understanding, deliverables, cost, timeline, notes, and next steps.
- Uses grounding rules to prevent unsupported claims.

The application also includes Pydantic input validation, Python quality checks, error handling, logging, a FastAPI endpoint, and a human approval checkpoint.

The external data source is `services.csv`, which contains verified freelance service information.

3. Framework Choice

CrewAI was selected because the problem naturally fits role-based multi-agent collaboration. The workflow contains three clearly separated responsibilities: research, scope analysis, and proposal writing. CrewAI provides a simple mechanism for assigning these responsibilities to specialized agents and passing task context between them.

LangGraph was not necessary because the workflow does not require complex branching, persistent graph state, or extensive control-flow management. CrewAI therefore provides an appropriate balance between implementation simplicity and multi-agent structure.

Python and FastAPI were used alongside CrewAI for validation, application logic, logging, and API deployment.

4. Evaluation Results

The system was evaluated using 8 varied test cases, including normal requests, an edge case with missing budget/deadline information, and an adversarial request.

Five evaluation criteria were defined:

Criterion	Result
Task Success	8/8
Factual Accuracy	8/8
Requirement Accuracy	6/8
Proposal Quality	8/8
Safety / Grounding	0/8 fully grounded

All 8 test cases completed successfully using the OpenRouter model configuration.

The system consistently identified the correct service, verified price, and delivery time. It also produced complete and professionally structured proposals.

However, the evaluation revealed a significant grounding problem. The agents frequently added unsupported features or business terms such as responsive design, SEO, admin panels,

deployment, databases, documentation, contracts, invoices, or deposits even when these were not included in the client request or verified service data.

The adversarial test demonstrated partial success: the system correctly avoided accepting an unsupported 50% discount and an unrealistic SEO guarantee, but it still introduced additional e-commerce features that were not explicitly supported.

5. Known Limitations

The main limitation is scope hallucination / unsupported scope expansion. Large language models have a tendency to add common industry features when generating professional proposals, even when those features were not requested.

Other limitations include:

- The service database is currently a local CSV file rather than a production database.
- Token usage depends on the selected model/provider and may not always be exposed consistently.
- Model/API failures can still occur because the system depends on an external LLM provider.
- Human review is still required for consequential client-facing output.
- Evaluation was performed on a small set of 8 test cases.
- The current system does not maintain long-term client history or persistent project state.
- Authentication, authorization, and production-grade API security would be required before public deployment.

6. Recommended Next Steps

Scaling

Replace the CSV service database with a production database and add persistent client/project records. The FastAPI service can then be deployed behind an authenticated API and scaled according to request volume.

The agent system can also be extended with additional tools such as CRM lookup, project history, proposal templates, and scheduling systems.

Guardrails

The proposal-generation stage should use strict closed-world grounding. Every proposed deliverable should be traceable to either the client's request or verified service data.

The improved Proposal Writer prompt implements this approach by requiring unsupported information to be marked as **“Not specified in the client request”** or requiring clarification rather than converting missing information into a promised feature.

Future versions should also add automated checks that compare generated proposal claims against the source requirements before approval.

Human Oversight

Human approval should remain mandatory before a proposal is sent to a client. The reviewer should verify price, timeline, deliverables, assumptions, and any requirements requiring clarification.

A future version could provide a structured review screen highlighting unsupported claims and potential scope risks.

Continuous Evaluation

The 8-case evaluation suite should become a regression test set. It should be re-run whenever the model, prompts, agents, tools, or service database are changed. Additional adversarial cases should be added as new failure patterns are discovered.

Conclusion

The project demonstrates a complete production-oriented multi-agent workflow rather than a simple chatbot. CrewAI coordinates specialized agents, Python provides validation and control logic, FastAPI exposes the system as an API, logging provides operational visibility, and human approval prevents fully autonomous client-facing actions.

The evaluation showed that the system can reliably identify services and produce professional proposals, while also revealing a clear weakness in grounding. The recommended closed-world prompting, automated claim checking, stronger guardrails, and continued human oversight provide a practical path toward a more reliable production system.